Table 6.3 Making a CORS request with an OAuth2 token.

Preflight request	Preflight response

`OPTIONS /api/posts HTTP/1.1`
`Host: 127.0.0.1:9999`
`Origin: http://localhost:1111`
`Access-Control-Request-Method: GET`
`Access-Control-Request-Headers: Authorization`
	`HTTP/1.1 204 No Content`
	`Access-Control-Allow-Origin: http://localhost:1111`
	`Access-Control-Allow-Methods: GET, POST, PUT, DELETE`
	`Access-Control-Allow-Headers: Authorization`

Actual request	**Actual response**

`GET /api/posts HTTP/1.1`
`Host: 127.0.0.1:9999`
`Origin: http://localhost:1111`
`Authorization: Bearer <OAUTH2 TOKEN>`
	`HTTP/1.1 200 OK`
	`Date: Sat, 05 Apr 2014 20:28:05 GMT`
	`Content-type: application/json; charset=UTF-8`
	`Content-length: 123`
	
	`{`
	`  "data": "..."`
	`}`

The node-oauth project has a popular OAuth2 implementation; it can be found at https://github.com/ciaranj/node-oauth. The OAuth2 website at http://oauth.net/2/ has more information on how OAuth2 works, as well as pointers to various server implementations. *Getting Started with OAuth2* by Ryan Boyd (O'Reilly Media, 2012) is also a great resource for learning how OAuth2 works.

This section introduced a lot of different security concepts. Here is a recap of each of these concepts, and how they're used:

- *Origin header*—Identifies where the client's request originates. Should only be used to verify that cross-origin requests from the origin are allowed.
- *Authorization cookies*—Can be used to identify the user making the request to the server. Cookies are always included on same-origin requests, and must be opted into by both the server and the client on cross-origin requests. But once opted in to, cookies can be vulnerable to CSRF attacks.
- *CSRF token*—Guards against CSRF attacks. Ensures that a request is coming from the user, not from a different page that is trying to trick the user into making a request.
- *Authorization header*—Used to add OAuth2 authorization information to the request. Authorizes a client to use a third-party API on behalf of a user, without the user giving their third-party username and password to the client.

Next, let's turn our attention to best practices for handling preflight requests.